

UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE ASIGNATURA

GRADO EN INGENIERÍA DE COMPUTADORES

Programación de Hardware Reconfigurable

Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3 Artix-7

Desarrollado por: Ander Regidor Barrante

Iván Serrano Prieto

Maria José García-Prieto Castells

Elena Esparza Baña

Dirigido por: Vicente A. García Alcántara

Madrid, 18 de junio de 2026



Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3 Artix-7

Desarrollado por: Ander Regidor Barrante

Iván Serrano Prieto

Maria José García-Prieto Castells

Elena Esparza Baña

Dirigido por: Vicente A. García Alcántara

Proyecto Fin de Asignatura, 18 de junio de 2026

E.T.S. de Ingeniería de Sistemas Informáticos

Campus Sur UPM, Carretera de Valencia (A-3), km. 7

28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{2026AnderRegidorBarranteIvnSerranoPrietoMariaJosGarcaPrietoCastellsElenaE
  title = {Implementación del procesador Intel 4004 en VHDL sobre FPGA Basys 3
Artix-7},
  type = {Bachelor's Thesis},
  author = {Regidor Barrante, A.; Serrano Prieto, I.; García-Prieto Castells,
M. J.; Esparza Baña, E.},
  school = {E.T.S. de Ingeniería de Sistemas Informáticos},
  year = {2026},
  month = {6},
}
```

Esta obra está bajo una licencia [Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional»](https://creativecommons.org/licenses/by-nc-sa/4.0/). Obra derivada de <https://github.com/blazaid/UPM-Report-Template>.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Resumen

Este trabajo recoge el diseño e implementación en VHDL del procesador CPU Intel 4004, el primer microprocesador comercial de la historia, sobre una placa de desarrollo Basys 3 equipada con una FPGA Artix-7 de Xilinx.

El diseño sigue la especificación técnica original del Intel 4004, organizando el hardware en bloques funcionales equivalentes a los del diagrama oficial: ruta de datos, unidad de control, pila de direcciones, banco de registros y lógica de interfaz con memoria externa. Se implementa un subconjunto funcional del juego de instrucciones suficiente para ejecutar una calculadora BCD de cuatro operaciones controlada mediante interruptores y visualizada en los displays de siete segmentos de la Basys 3.

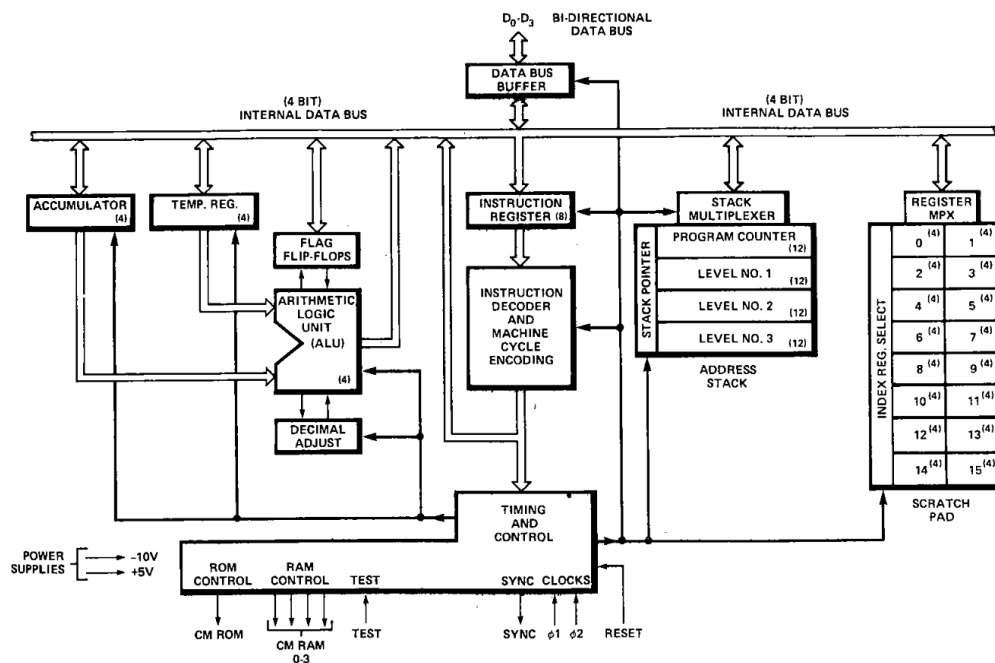


Diagrama de bloques del procesador Intel 4004

Palabras clave: VHDL; Intel 4004; FPGA; Artix-7; Arquitectura de computadores

Índice general

1	Introducción	1
1.1	Objetivos	1
1.2	Alcance de la implementación	1
1.3	Estructura de la memoria	2
2	Estado de la cuestión	3
2.1	El procesador Intel 4004	3
2.2	Herramientas utilizadas	3
3	Metodología	4
4	Diseño	5
4.1	Filosofía de diseño y paquete global	5
4.2	Primitivas de hardware	7
4.3	Ruta de datos	8
4.4	Pila de direcciones	13
4.5	Bus interno y buffer de datos	15
4.6	Unidad de control	16
4.7	Integración: <i>top-level</i> de la CPU	18
4.8	Memoria de programa y SoC para la Basys 3	19

Índice general	II
4.9 Traza de ejecución de instrucciones representativas	21
4.10 Limitaciones respecto a la especificación	26
5 Verificación	27
5.1 Testbench del multiplexor de bus interno	27
5.2 Testbench del sistema completo	27
6 Implementación	28
7 Conclusiones	29
A Listado de ficheros del proyecto	30
Glosario	32
Siglas	32

Índice de tablas

4.1	Puertos de la entidad <code>d_ff_en</code>	7
4.2	Puertos del <code>registro_4b</code> (análogos en <code>registro_12b</code>)	8
4.3	Puertos de la entidad <code>flipflopJK</code>	8
4.4	Puertos de la entidad <code>accumulator</code>	9
4.5	Puertos de la entidad <code>instruction_register</code>	10
4.6	Puertos de la entidad <code>flag_flip_flops</code>	11
4.7	Puertos de la entidad <code>alu</code>	11
4.8	Operaciones de la ALU	12
4.9	Puertos de la entidad <code>decimal_adjust</code>	12
4.10	Puertos de la entidad <code>scratch_pad</code>	13
4.11	Puertos de la entidad <code>stack_3L</code>	14
4.12	Fuentes del <code>bus_interno_mux</code> (cada una con su señal <code>_oe</code>)	15
4.13	Puertos de la entidad <code>data_bus_buffer</code>	16
4.14	Puertos de la entidad <code>instruction_decoder</code> conectados en <code>cpu_4004_top</code>	17
4.15	Fases del ciclo de máquina y función de cada una	18
4.16	Puertos de la entidad <code>ROM</code>	19
4.17	Puertos de la entidad <code>basys3_4004_soc</code>	21
4.18	Traza ciclo a ciclo de <code>ADD R1 (ACC=3, R1=5, C=0)</code>	23
4.19	Traza de <code>JCN Z, 0x2A</code> — ciclo 1 (fetch del opcode)	24

4.20	Traza de JCN Z, 0x2A — ciclo 2 (segundo byte y resolución del salto) .	25
4.21	Funcionalidad no implementada respecto a la especificación original .	26
A.1	Ficheros fuente VHDL del proyecto	31

Índice de listados

4.1	Contenido del paquete <code>pkg_4004</code>	5
4.2	Serialización del PC en <code>mtx_stack</code>	14

1.

Introducción

El Intel 4004, lanzado en noviembre de 1971, fue el primer [Unidad Central de Proceso \(CPU\)](#) comercial integrado en un único circuito integrado. Diseñado para la calculadora de escritorio Busicom 141-PF, estableció las bases de la industria del microprocesador con apenas 2 300 transistores, una arquitectura de 4 bits y una frecuencia máxima de 740 kHz.

El objetivo de este proyecto, enmarcado en la asignatura [Programación de Hardware Reconfigurable \(PHR\)](#) del Grado en Ingeniería de Computadores de la *Escuela Técnica Superior de Ingeniería de Sistemas Informáticos* de la *Universidad Politécnica de Madrid*, es reimplementar este procesador en [VHDL](#) y sintetizarlo sobre una [FPGA](#) Artix-7 Basys 3.

1.1. Objetivos

1. Estudiar la especificación técnica original del Intel 4004.
2. Diseñar la arquitectura en [VHDL](#) dividida en bloques funcionales fieles al diagrama de la especificación.
3. Implementar un subconjunto funcional del juego de instrucciones suficiente para ejecutar la aplicación de demostración.
4. Verificar el diseño mediante simulación con testbenches.
5. Sintetizar e implementar el diseño sobre la Basys 3 Artix-7.

1.2. Alcance de la implementación

El Intel 4004 original define 46 instrucciones. Esta implementación cubre el subconjunto necesario para la aplicación de demostración: aritmética (ADD, SUB, INC, DAA), carga de registros (LD, LDM, XCH, FIM), control de flujo (JUN, JMS, JCN, BBL), acceso al banco de registros (SRC), entrada/salida con ROM (RDR, WRR) y manipulación de flags

(CLC, CMC, CLB). Quedan fuera del alcance las instrucciones de acceso a la RAM 4002 y otros grupos de control; véase la sección 4.10.

1.3. Estructura de la memoria

El capítulo 2 describe la arquitectura del Intel 4004. El capítulo 3 presenta la metodología seguida. El capítulo 4 contiene el núcleo técnico del trabajo, recorriendo los bloques VHDL en orden de dependencia ascendente: desde las primitivas hasta el integrador top-level. Los capítulos 5 e 6 recogen la verificación y la síntesis sobre hardware.

2. Estado de la cuestión

2.1. El procesador Intel 4004

El Intel 4004 fue diseñado por Federico Faggin, Marcian Hoff, Stanley Mazor y Masatoshi Shima. Sus características principales son:

- Bus de datos externo bidireccional de 4 bits (D0–D3), multiplexado en el tiempo para transportar tanto datos como los tres **nibble** de la dirección de 12 bits.
- Acumulador de 4 bits y su **ALU** asociada.
- Banco de 16 registros internos (**scratchpad**) de 4 bits, agrupables en 8 pares de 8 bits.
- **Pila de direcciones** de tres niveles de 12 bits; el **PC** es siempre la cima de la pila.
- Ciclo de máquina de 8 fases (A1–A3, M1–M2, X1–X3) controlado por dos relojes en cuadratura no solapados (ϕ_1 , ϕ_2).
- Señales de control: SYNC, CM-ROM y CM-RAM[3 : 0].
- Juego de 46 instrucciones de 8 bits (16 bits para instrucciones de dos ciclos de máquina).

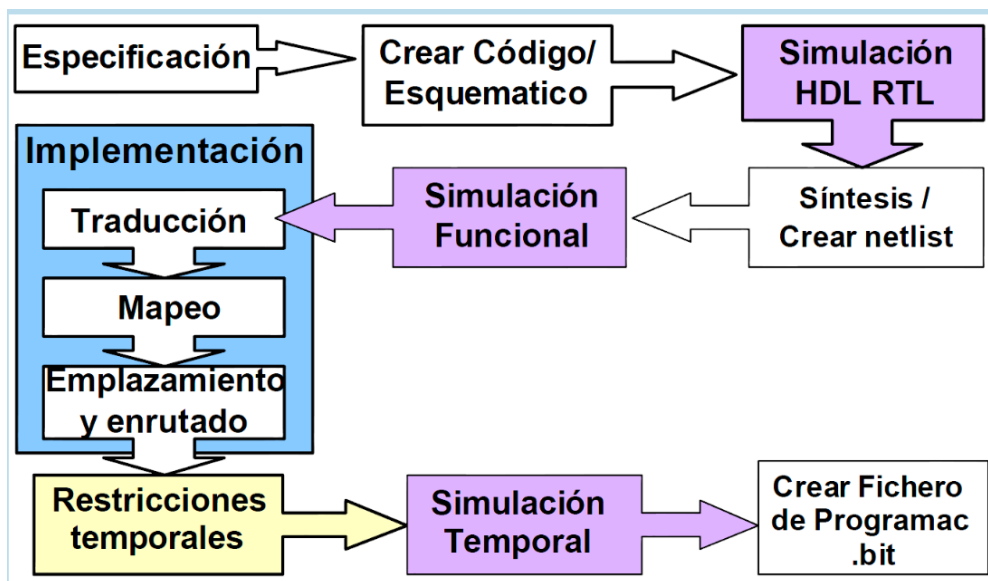
2.2. Herramientas utilizadas

- *Vivado Design Suite* 2024 de Xilinx/AMD.
- **VHDL**-93 como lenguaje de descripción hardware.
- Placa Basys 3 (Artix-7 XC7A35TICPG236-1L) como destino hardware.

3.

Metodología

El proyecto sigue la metodología de diseño de la asignatura [PHR](#):



Fases de la metodología de diseño de la asignatura [PHR](#)

El proceso comenzó con el análisis de la especificación técnica del Intel 4004, identificando los bloques funcionales y el subconjunto de instrucciones a implementar. A continuación se diseñó la jerarquía de ficheros [VHDL](#), un fichero por bloque funcional. Cada bloque se verificó de forma individual mediante testbench antes de integrarse en el nivel superior. Por último, se sintetizó e implementó el diseño completo sobre la Basys 3.

Este capítulo recorre los bloques **VHDL** del diseño en orden de dependencia ascendente: comenzando por las constantes globales y las primitivas de hardware, avanzando a través de la ruta de datos, la pila, el bus y la unidad de control, hasta llegar al fichero integrador top-level y al **SoC** que lo conecta con la placa física.

4.1. Filosofía de diseño y paquete global

4.1.1. Arquitectura estructural jerárquica

La primera decisión del proyecto fue adoptar una arquitectura puramente estructural en todos los niveles de la jerarquía: cada bloque funcional instancia componentes de nivel inferior hasta llegar, en la base, a primitivas de *Flip-Flop* (**FF**) individuales.

Esta elección responde a tres criterios. Primero, la síntesis es completamente predecible: cada `registro_4b` se convierte en cuatro primitivos **FDRE** de la Artix-7, sin inferencias implícitas. Segundo, la verificación es incremental: cada primitiva puede simularse de forma aislada antes de integrarse en el bloque superior. Tercero, existe una correspondencia directa entre el diagrama de bloques de la especificación y la jerarquía de ficheros **VHDL**, facilitando la trazabilidad.

4.1.2. Paquete de constantes

Fichero: `pkg_4004.vhd`

El paquete `pkg_4004` centraliza las constantes globales. Todos los módulos lo importan con `use work.pkg_4004.all`.

Listado 4.1. Contenido del paquete `pkg_4004`

```
constant BUS_W      : integer := 4;
constant BUS_CERO   : std_logic_vector(BUS_W-1 downto 0) := "
    0000";
constant BUS_ERROR   : std_logic_vector(BUS_W-1 downto 0) := "
    XXXX";
```

BUS_W define el ancho del **bus interno**; todos los vectores de datos se dimensionan con ella. BUS_ERROR sirve para detectar colisiones de bus en simulación: si dos fuentes conducen el bus simultáneamente, la salida del multiplexor central toma el valor "XXXX", inmediatamente visible en el visor de señales de Vivado.

4.2. Primitivas de hardware

En la base de la jerarquía se encuentran tres primitivas que representan los elementos de memoria más simples.

4.2.1. *Flip-flop* D con enable

Fichero: `d_ff_en.vhd`

Es el elemento de memoria fundamental del diseño. Prácticamente todos los registros de la **CPU** se construyen como matrices de instancias de esta entidad. Implementa un *Flip-Flop* (FF) tipo D síncrono con reset asíncrono activo alto y señal de habilitación.

Tabla 4.1. Puertos de la entidad `d_ff_en`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj; captura en flanco de subida
<code>reset</code>	in	1	Reset asíncrono activo alto
<code>en</code>	in	1	Habilitación síncrona
<code>d</code>	in	1	Dato de entrada
<code>q</code>	out	1	Salida del FF

La arquitectura `Behavioral` implementa el proceso sensible a `clk` y `reset`: si `reset` está activo, la salida se lleva a '0' de forma asíncrona; en el flanco de subida, si `en` = '1', se captura `d`. El sintetizador mapea este patrón directamente sobre un primitivo `FDRE` de la Artix-7.

4.2.2. Registros de 4 y 12 bits

Fichero: `registro_4b.vhd`, `registro_12b.vhd`

Ambos se implementan con una sentencia `GENERATE` que instancia N copias de `d_ff_en` en paralelo ($N = 4$ ó $N = 12$).

Tabla 4.2. Puertos del `registro_4b` (análogos en `registro_12b`)

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>R</code>	in	1	Reset asíncrono activo alto
<code>E</code>	in	1	Enable síncrono de escritura
<code>D_in</code>	in	4 (12)	Dato a cargar cuando <code>E = '1'</code>
<code>Q_out</code>	out	4 (12)	Dato almacenado

4.2.3. *Flip-flop* JK

Fichero: `flipflopJK.vhd`

FF de tipo JK utilizado exclusivamente en el puntero de pila (sección 4.4). Proporciona las salidas directa (`q`) y complementada (`notq`).

Tabla 4.3. Puertos de la entidad `flipflopJK`

Puerto	Dir.	Bits	Descripción
<code>j</code>	in	1	Entrada J
<code>k</code>	in	1	Entrada K
<code>r</code>	in	1	Reset asíncrono activo alto
<code>clk</code>	in	1	Reloj de captura (flanco de subida)
<code>q</code>	out	1	Salida directa
<code>notq</code>	out	1	Salida complementada

4.3. Ruta de datos

4.3.1. Acumulador

Fichero: `accumulator.vhd`

El acumulador almacena el primer operando y el resultado de toda operación aritmética o lógica. Se implementa como un *wrapper* sobre `registro_4b`: la entidad `accumulator`

no añade lógica propia, ya que el acumulador del Intel 4004 no tiene circuitería especial asociada. Su existencia como entidad separada responde a razones de legibilidad y trazabilidad con la especificación.

Tabla 4.4. Puertos de la entidad `accumulator`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_en</code>	in	1	Señal de escritura desde <code>timing_and_control</code>
<code>d_in</code>	in	4	Dato a cargar (del bus interno)
<code>q_out</code>	out	4	Contenido actual (hacia la ALU y el bus)

4.3.2. Registro temporal

Fichero: `temp_register.vhd`

El registro temporal almacena el segundo operando de la [ALU](#). Durante la fase X1, el controlador carga en él el valor del registro fuente del [scratchpad](#); en la fase X2, ese valor pasa a la entrada B de la [ALU](#). Su implementación es idéntica a la del acumulador: un *wrapper* sobre `registro_4b`.

4.3.3. Registro de instrucción

Fichero: `instruction_register.vhd`

El **Registro de Instrucción (IR)** captura la instrucción de dos **nibble** procedentes del **bus interno** durante las fases de búsqueda. Se implementa de forma estructural como ocho instancias de `d_ff_en` organizadas en dos grupos de cuatro mediante `GENERATE`: el grupo alto (bits 7–4) se captura en la fase M1 cuando `load_high = '1'`; el grupo bajo (bits 3–0) en la fase M2 cuando `load_low = '1'`.

Tabla 4.5. Puertos de la entidad `instruction_register`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_high</code>	in	1	Carga nibble alto (activo en M1)
<code>load_low</code>	in	1	Carga nibble bajo (activo en M2)
<code>bus_in</code>	in	4	Dato del bus interno
<code>instr_out</code>	out	8	Instrucción completa hacia el decodificador

La señal `disable_ir` (gestionada en `cpu_4004_top`) impide que el **IR** capture un nuevo opcode durante el segundo ciclo de instrucciones de 2 bytes, preservando el opcode del primer byte durante la ejecución.

4.3.4. Registros de flags

Fichero: `flag_flip_flops.vhd`

Almacena el vector de estado de la **CPU**. La lógica de formación de la entrada (`d_in`) se describe en `cpu_4004_top`:

- Bit 0, Carry: recibe el acarreo de la **ALU** en operaciones aritméticas, '0' en CLC y CLB, o el complemento del carry actual en CMC.
- Bit 1, Test: conectado directamente al pin físico `test_pin`.
- Bits 2–3: reservados a '0'.

Tabla 4.6. Puertos de la entidad `flag_flip_flops`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>reset</code>	in	1	Reset asíncrono
<code>load_en</code>	in	1	Señal de escritura
<code>d_in</code>	in	4	Valor a cargar
<code>q_out</code>	out	4	Estado actual de los flags
<code>flags_oe</code>	in	1	OE hacia el bus interno

4.3.5. Unidad Aritmético-Lógica

Fichero: `alu.vhd`

La **Unidad Aritmético-Lógica (ALU)** realiza todas las operaciones aritméticas y lógicas. Su arquitectura `Combinational` no contiene ningún elemento de memoria; el resultado está disponible de forma continua.

Tabla 4.7. Puertos de la entidad `alu`

Puerto	Dir.	Bits	Descripción
<code>A_in</code>	in	4	Primer operando (acumulador o registro de scratchpad)
<code>B_in</code>	in	4	Segundo operando (temp. register o ajuste decimal)
<code>carry_in</code>	in	1	Acarreo de entrada (<code>flags_out[0]</code>)
<code>op</code>	in	3	Código de operación
<code>result_out</code>	out	4	Resultado
<code>carry_out</code>	out	1	Acarreo de salida
<code>zero_flag</code>	out	1	Activo si <code>result_out = "0000"</code>

Tabla 4.8. Operaciones de la ALU

op	Operación	Descripción
000	ADD	$A + B + C_{in}$
001	SUB	$A + \overline{B} + C_{in}$ (complemento a 1, fiel al 4004)
010	AND	$A \text{ AND } B$
011	OR	$A \text{ OR } B$
100	XOR	$A \text{ XOR } B$
101	PASS	Pasa A sin modificar (valor por defecto)
110	INC	$A + 1$

Los tres resultados de mayor coste (ADD, SUB, INC) se calculan siempre en paralelo mediante señales intermedias de 5 bits, y un proceso `case final` selecciona cuál se propaga a la salida. El bit de acarreo es el bit 4 de la señal de 5 bits correspondiente.

La entrada B de la ALU proviene de un multiplexor en `cpu_4004_top`: normalmente conecta con `temp_register`, pero cuando `ctrl_alu_b_mux = '1'` conecta con la salida del bloque de ajuste decimal.

4.3.6. Ajuste decimal

Fichero: `decimal_adjust.vhd`

Bloque combinacional que calcula la corrección BCD para DAA. Si el acarreo está activo o el acumulador supera 9, la corrección es 0110 (+6); en caso contrario es 0000.

Tabla 4.9. Puertos de la entidad `decimal_adjust`

Puerto	Dir.	Bits	Descripción
<code>acc_in</code>	in	4	Contenido actual del acumulador
<code>carry_in</code>	in	1	Acarreo actual
<code>adj_out</code>	out	4	Corrección a sumar (0000 ó 0110)

4.3.7. Banco de registros (*scratchpad*)

Fichero: `scratch_pad.vhd`, `mtx_scratchpad.vhd`

El [scratchpad](#) implementa los 16 registros de propósito general de 4 bits del Intel 4004 (R0–R15), agrupables en 8 pares de 8 bits. La arquitectura `Structural` instancia 16 copias de `registro_4b` y un multiplexor 16:1 (`mtx_scratchpad`) para leer el registro seleccionado.

Tabla 4.10. Puertos de la entidad `scratch_pad`

Puerto	Dir.	Bits	Descripción
<code>clk_r</code>	in	1	Reloj del sistema
<code>rst_r</code>	in	1	Reset asíncrono
<code>w_e</code>	in	1	Write enable
<code>pair_mode</code>	in	1	'1': acceso en par de 8 bits
<code>address</code>	in	4	Dirección del registro (0–15)
<code>data_in</code>	in	8	Dato a escribir
<code>data_out</code>	out	8	Dato leído

En modo par (`pair_mode = '1'`), se activan simultáneamente el registro par e impar: el par recibe `data_in[7:4]` y el impar `data_in[3:0]`, modelando las instrucciones FIM (carga de un par con inmediato de 8 bits) y SRC (envío de dirección de 8 bits al bus externo en dos [nibble](#)).

El direccionamiento durante instrucciones de par se gestiona en `cpu_4004_top`: la lógica selecciona el registro par o impar según la fase del ciclo de máquina (`t_state`).

4.4. Pila de direcciones

Fichero: `stack_3L.vhd`

La [pila de direcciones](#) implementa los tres niveles de 12 bits del Intel 4004. A diferencia de un diseño convencional donde el **PC** es un registro separado, en el 4004 el **PC** es siempre la cima de la pila. Se implementa como cuatro registros de 12 bits (`registro_12b`) con un puntero de pila de 2 bits que selecciona el nivel activo.

Tabla 4.11. Puertos de la entidad `stack_3L`

Puerto	Dir.	Bits	Descripción
<code>Clk_s</code>	in	1	Reloj del sistema
<code>sp_clk</code>	in	1	Reloj del puntero (activo solo en JMS y BBL)
<code>U_D_s</code>	in	1	Dirección: '0' push (JMS), '1' pop (BBL)
<code>D_12_s</code>	in	12	Dirección a escribir (PC+1 o dirección de salto)
<code>E_s</code>	in	1	Enable de escritura
<code>R_s</code>	in	1	Reset
<code>oe_s</code>	in	1	OE hacia el bus interno
<code>fase_s</code>	in	2	Fase del reloj para serialización
<code>sal_4_stck</code>	out	4	nibble serializado hacia el bus interno
<code>PC_out</code>	out	12	Valor actual del PC

La entrada de datos (`stack_in_12b`) se decide en `cpu_4004_top`: si la instrucción es un salto efectivo se carga la dirección destino; en caso contrario se carga `PC + 1`.

Puntero de pila

Fichero: `puntero_stack.vhd`

Contador bidireccional de 2 bits construido con dos [FF](#) tipo JK (`flipflopJK`). Cuando `up_down = '1'` el puntero avanza (push en JMS); cuando es '0' retrocede (pop en BBL). El reloj del puntero (`sp_clk`) solo conmuta durante instrucciones de llamada o retorno.

Multiplexor de pila con serialización

Fichero: `mtx_stack.vhd`

Selecciona el nivel activo y serializa su contenido de 12 bits en tres [nibble](#) de 4 bits para volcarlo al [bus interno](#) durante A1, A2 y A3:

Listado 4.2. Serialización del PC en `mtx_stack`

```

when "00" => y <= reg_activo(3 downto 0);    -- A1: nibble bajo
when "01" => y <= reg_activo(7 downto 4);    -- A2: nibble
        medio
when "10" => y <= reg_activo(11 downto 8);    -- A3: nibble alto

```

4.5. Bus interno y buffer de datos

4.5.1. Bus interno — multiplexor central

Fichero: bus_interno_mux.vhd

El **bus interno** de 4 bits es el canal de comunicación entre todos los bloques funcionales. Se implementa como un multiplexor centralizado en lugar de un bus triestado interno.

En **VHDL** lo más natural sería usar señales `std_logic` con alta impedancia 'Z'. Sin embargo, los recursos internos de la Artix-7 no disponen de buffers triestado en la *fabrica*; el sintetizador los convertiría igualmente en multiplexores, pero de forma implícita y menos predecible. El multiplexor explícito ofrece además detección de colisiones en simulación mediante `BUS_ERROR = "XXXX"`.

Tabla 4.12. Fuentes del bus_interno_mux (cada una con su señal `_oe`)

Fuente	Descripción
acc_out	Acumulador
tmp_reg_out	Registro temporal
flags_out	Registros de flags
alu_out	Resultado de la ALU
decoder_out	Decodificador (inmediatos LDM/BBL)
stack_out	Pila de direcciones (PC serializado)
reg_mux_out	<i>Scratchpad</i>
ext_buf_out	Buffer de bus externo (instrucción de la ROM)

4.5.2. Buffer de bus de datos

Fichero: `data_bus_buffer.vhd`

Gestiona la comunicación bidireccional con el pin físico `D_bus` (D0–D3). Cuando `dir_out = '0'`, el CPU vuelca datos al exterior (fases A1–A3: la dirección del **PC** va hacia la ROM). Cuando `dir_out = '1'`, el pin externo alimenta el **bus interno** (fases M1–M2: la ROM entrega la instrucción).

Tabla 4.13. Puertos de la entidad `data_bus_buffer`

Puerto	Dir.	Bits	Descripción
<code>internal_bus_in</code>	<code>in</code>	4	Dato del bus interno (hacia el exterior)
<code>internal_bus_out</code>	<code>out</code>	4	Dato hacia el bus interno (desde el exterior)
<code>dir_out</code>	<code>in</code>	1	'1': CPU escribe en <code>D_bus</code>
<code>data_bus_ext</code>	<code>inout</code>	4	Pin bidireccional D0–D3

A diferencia del bus interno, aquí el triestado es correcto y necesario: `data_bus_ext` es un pin físico del encapsulado, y los pines `I0B` de la Artix-7 sí disponen de buffers triestado reales.

4.6. Unidad de control

4.6.1. Decodificador de instrucciones

Fichero: `instruction_decoder.vhd`

El decodificador traduce el byte de instrucción almacenado en el **IR** en señales de control combinacionales para la unidad de temporización.

Tabla 4.14. Puertos de la entidad `instruction_decoder` conectados en `cpu_4004_top`

Puerto	Dir.	Bits	Descripción
<code>ir_in</code>	in	8	Byte de instrucción actual
<code>inst_group</code>	out	16	Codificación One-Hot del grupo de instrucción
<code>is_2byte</code>	out	1	Instrucción de 2 ciclos (JUN, JMS, JCN, FIM)
<code>bus_out</code>	out	4	Dato inmediato (nibble bajo, para LDM y BBL)
<code>out_en</code>	out	1	OE del inmediato hacia el bus interno

La entidad define además puertos indicadores individuales (`is_daa`, `is_tcs`, `is_rdr`, `is_wrr`, `is_wmp`, `is_clc`, `is_clb`, `is_cmc`), pero en `cpu_4004_top` no se conectan: el bloque `timing_and_control` infiere esa misma información combinando la señal `inst_group` con el contenido de `ir_in`, que recibe directamente, por lo que los indicadores separados resultan redundantes a nivel de integración.

La decodificación principal se realiza con un `case` sobre el nibble alto del opcode (`ir_in[7:4]`), generando una codificación One-Hot de 16 bits en `inst_group`: el bit n activo indica que la instrucción pertenece al grupo con opcode alto n .

Las instrucciones LDM (grupo 13) y BBL (grupo 7) colocan su dato inmediato directamente en `bus_out` (`ir_in[3:0]`) y activan `out_en` para que el valor aparezca en el **bus interno** sin ningún registro adicional.

4.6.2. Temporización y control

Fichero: `timing_and_control.vhd`

El bloque `timing_and_control` es el corazón de la unidad de control. Implementa la *Finite State Machine* (**FSM**) de 8 estados del ciclo de máquina y genera todas las señales de control del *datapath*.

La **FSM** se describe con arquitectura `Behavioral`: un proceso secuencial incrementa un contador de 3 bits en cada flanco de subida de `clk_ph1`, ciclando de 000 a 111. Un segundo proceso combinacional produce las señales de salida mediante un `case`.

Tabla 4.15. Fases del ciclo de máquina y función de cada una

Estado	Fase	Función
000	A1	<code>sync = '1'</code> ; nibble bajo del PC al bus externo
001	A2	Nibble medio del PC al bus externo
010	A3	Nibble alto del PC; activa <code>cm_rom</code>
011	M1	Captura nibble alto de la instrucción desde la ROM
100	M2	Captura nibble bajo de la instrucción desde la ROM
101	X1	Primera fase de ejecución (lectura de operando)
110	X2	Segunda fase de ejecución (operación ALU / escritura)
111	X3	Tercera fase de ejecución (actualización del PC)

Las señales `inst_group` y `current_frag` procedentes del decodificador determinan qué señales de control se activan durante X1–X3. La señal `current_frag` distingue el primer del segundo ciclo de máquina de instrucciones de 2 bytes.

4.7. Integración: *top-level* de la CPU

Fichero: `cpu_4004_top.vhd`

El fichero `cpu_4004_top.vhd` ensambla todos los bloques descritos en las secciones anteriores mediante cláusulas `port map`. No contiene lógica funcional propia salvo tres elementos de *glue logic* que no pertenecen a ningún bloque individual:

- **Bypass de INC:** la entrada A de la **ALU** se conecta normalmente al acumulador, pero durante `INC` se redirige a la salida del **scratchpad** para incrementar un registro de índice directamente, sin alterar el acumulador.
- **Fix de DAA:** el carry de entrada a la **ALU** se fuerza a '0' durante `DAA` para que el ajuste decimal no incluya el carry antiguo como sumando adicional.
- **Lógica de instrucciones de 2 bytes:** un proceso secuencial gestiona `current_frag` y `disable_ir` para controlar la ejecución en dos ciclos de `JUN`, `JMS`, `JCN` y `FIM`. Captura el segundo byte en `second_byte_reg` y evalúa la condición de salto para `JCN`

comparando el acumulador, el carry y el `test_pin` con las condiciones codificadas en el nibble bajo del opcode.

La entidad expone exactamente los pines físicos del DIP-16 del Intel 4004 original: `clk_ph1`, `clk_ph2`, `reset`, `test_pin`, `sync`, `cm_rom`, `cm_ram[3:0]` y el bus bidireccional `D_bus[3:0]`.

4.8. Memoria de programa y SoC para la Basys 3

4.8.1. ROM de programa: modelo del chip 4001

Fichero: `ROM.vhd`

La ROM modela el comportamiento de uno o varios chips Intel 4001 externos. En el chip real la ROM es externa al 4004 y se conecta al bus D0–D3. En este diseño la entidad ROM se instancia en `basys3_4004_soc` (no en `cpu_4004_top`) y comparte el bus bidireccional `bus_io`.

Tabla 4.16. Puertos de la entidad ROM

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Reloj del sistema
<code>cm_rom</code>	in	1	Chip select (activo alto, desde la CPU)
<code>fase</code>	in	2	Fase del bus
<code>bus_io</code>	inout	4	Bus D0–D3 compartido con la CPU
<code>rom0_io_in</code>	in	4	Puerto de E/S ROM 0 (Operando A)
<code>rom1_io_in</code>	in	4	Puerto de E/S ROM 1 (Operando B)
<code>rom2_io_in</code>	in	4	Puerto de E/S ROM 2 (Operador)
<code>rom2_io_out</code>	out	4	Salida ROM 2 (Resultado BCD)

La ROM contiene un array de 4096 bytes cargado con el programa de la calculadora. Su lógica monitoriza `fase` para capturar la dirección de 12 bits en tres fases (A1, A2, A3) y volcar el byte de instrucción al bus en M1 y M2.

Adicionalmente, implementa los puertos de E/S del chip 4001: `SRC` establece la ROM activa; `RDR` lee el puerto y vuelca su valor al bus; `WRR` captura el acumulador y lo escribe en el puerto de salida. Esto permite que la calculadora lea los operandos desde interruptores y escriba el resultado hacia el display.

El programa almacenado implementa las cuatro operaciones: suma, resta con saturación a cero, comparación (devuelve 1 si $A > B$, 2 si $A < B$, 3 si $A = B$) y multiplicación por sumas sucesivas.

4.8.2. Sistema en chip para la Basys 3

Fichero: `basys3_4004_soc.vhd`

La entidad `basys3_4004_soc` conecta la CPU y la ROM con los recursos físicos de la Basys 3. Sus responsabilidades son:

1. Generación de relojes: convierte el oscilador de 100 MHz en los dos relojes en cuadratura del 4004 a ≈ 740 kHz (divisor de 135 ciclos). Fase 1 activa en los primeros 31 ciclos; fase 2 centrada en la segunda mitad, sin solapamiento con la primera.
2. Debouncing del reset: el botón `btnC` se filtra con un registro de desplazamiento de 16 bits muestreado a 1 kHz. El reset se activa solo si el botón se mantiene pulsado durante 16 ms consecutivos.
3. Seguimiento del ciclo de máquina: un proceso local replica la [FSM](#) sincronizándose con `sync` para indicar a la ROM en qué fase se encuentra el bus.
4. Mapeo de E/S: `sw[3:0]` y `sw[7:4]` se conectan a los puertos de entrada de ROM 0 y ROM 1 (operandos A y B); `sw[9:8]` al puerto de ROM 2 (operador). La salida de ROM 2 (`rom2_io_out`) se muestra en el display.
5. Controlador de display: un contador de 20 bits multiplexa los cuatro dígitos a ≈ 200 Hz, mostrando operando A, símbolo del operador, operando B y resultado. La decodificación a siete segmentos se implementa con una función [VHDL](#) pura (`hex_to_7seg`).

Tabla 4.17. Puertos de la entidad `basys3_4004_soc`

Puerto	Dir.	Bits	Descripción
<code>clk</code>	in	1	Oscilador de 100 MHz de la Basys 3
<code>btnC</code>	in	1	Botón central (reset físico)
<code>sw</code>	in	10	[3:0] op. A, [7:4] op. B, [9:8] operador
<code>seg</code>	out	7	Segmentos del display (activo bajo)
<code>dp</code>	out	1	Punto decimal (fijo apagado)
<code>an</code>	out	4	Ánodos de los 4 dígitos (activo bajo)

4.9. Traza de ejecución de instrucciones representativas

Para ilustrar el funcionamiento conjunto de todos los bloques se detallan a continuación dos instrucciones representativas extraídas del programa de la calculadora: `ADD R1` (aritmética de un ciclo de máquina) y `JCN` (salto condicional de dos ciclos). Entre ambas cubren la ruta de datos aritmética completa, el mecanismo de instrucciones de 2 bytes y la lógica de actualización del contador de programa.

4.9.1. ADD R1 — instrucción aritmética de un ciclo

ADD R1 tiene el opcode 0×81 (nibble alto 1000, nibble bajo 0001). En el ejemplo se asume: acumulador = 3, R1 = 5, flag de carry = '0'. El resultado esperado es ACC = 8 con carry = '0'.

Tabla 4.18. Traza ciclo a ciclo de ADD R1 (ACC=3, R1=5, C=0)

Estado	Señales activas	Acción
A1	sync, stack_oe	La pila serializa PC[3:0] al bus D0-D3 (ext_dir='0'). La ROM latching el nibble bajo de la dirección (addr_reg[3:0]).
A2	stack_oe	La pila serializa PC[7:4]. La ROM latching el nibble medio (addr_reg[7:4]).
A3	stack_oe, cm_rom	La pila serializa PC[11:8]; chip select activado. La ROM ensambla la dirección completa y lee la instrucción: data_reg ← 0x81.
M1	cm_rom, ext_dir='1', load_ir_high	La ROM vuelca el nibble alto "1000" al bus. El buffer externo lo introduce en el bus interno. El IR captura IR[7:4] ← "1000".
M2	ext_dir='1', load_ir_low	La ROM vuelca el nibble bajo "0001" al bus. IR[3:0] ← "0001". El decodificador activa inst_group(8) (grupo ADD) e is_2byte='0'.
X1	scratch_oe, tmp_load	El scratchpad selecciona R1 (índice IR[3:0] = "0001") y vuelca el valor 5

El camino completo del datapath aritmético queda así: el scratchpad expone el operando en el bus interno durante X1; el registro temporal lo captura; durante X2 la ALU opera con el acumulador y el registro temporal como entradas, y vuelca el resultado al mismo bus; el acumulador y los flags se actualizan en el flanco ascendente de `clk_ph1` que cierra X2. El conjunto de X1 y X2 basta para completar la aritmética en un único ciclo de máquina.

4.9.2. JCN — salto condicional de dos ciclos

JCN (opcode `0x1X`) es una instrucción de 2 bytes: el primer byte codifica la condición en su nibble bajo; el segundo byte contiene la dirección de destino dentro de la página actual.

El ejemplo usa la instrucción en la posición `0x018` del programa (JCN Z, `0x2A`, opcode `0x14`): si el acumulador es cero salta a `0x02A`; si no, continúa en `0x01A`.

Tabla 4.19. Traza de JCN Z, `0x2A` — ciclo 1 (fetch del opcode)

Estado	Señales activas	Acción
A1-A3	<code>stack_oe</code> , <code>cm_rom</code>	La pila vuelca <code>PC = 0x018</code> en tres nibbles. La ROM ensambla la dirección y lee <code>0x14</code> .
M1-M2	<code>ext_dir='1'</code> , <code>load_ir_high/low</code>	<code>IR ← 0x14</code> . El decodificador activa <code>inst_group(1)</code> (grupo JCN) y aserta <code>is_2byte='1'</code> .
X1-X2	—	JCN no opera el datapath en el primer ciclo; ninguna señal de registro queda activa.
X3	<code>stack_en</code>	<code>current_frag='0'</code> : <code>stack_in = PC+1 = 0x019</code> ; el contador de programa apunta al segundo byte. El proceso secuencial detecta <code>is_2byte='1'</code> y transiciona <code>current_frag ← '1'</code> y <code>disable_ir ← '1'</code> para el ciclo siguiente.

Tabla 4.20. Traza de JCN Z, 0x2A — ciclo 2 (segundo byte y resolución del salto)

Estado	Señales activas	Acción
A1-A3	stack_oe, cm_rom	La pila vuelca PC = 0x019. La ROM lee el segundo byte 0x2A.
M1	ext_dir='1'(disable_ir='1')	La ROM vuelca "0010" al bus. disable_ir='1' bloquea la carga del IR; IR conserva 0x14 (opcode del primer byte). El proceso de captura almacena second_byte_reg[7:4] $\leftarrow$ "0010".
M2	ext_dir='1'(disable_ir='1')	La ROM vuelca "1010" al bus. second_byte_reg[3:0] $\leftarrow$ "1010"; second_byte_reg = 0x2A.
X1-X2	—	La lógica combinacional evalúa la condición de salto: cond_accum = (ACC = "0000") (IR[2]='1'); cond_carry = '0' (IR[1]='0'); cond_test = '0' (IR[0]='0'). jcn_taken = cond_met (sin inversión, IR[3]='0'). target_jump_addr = PC[11:8] & 0x2A = 0x02A.
X3	stack_en	Si ACC = 0 (jcn_taken='1'): stack_in = 0x02A $\Rightarrow$ PC $\leftarrow$ 0x02A. Si ACC $\neq$ 0 (jcn_taken='0'): stack_in = PC+1 = 0x01A $\Rightarrow$ ejecución continúa. current_frag y disable_ir vuelven a '0'.

La instrucción JCN pone de manifiesto tres mecanismos que ADD no involucra. Primero, la señal disable_ir preserva el opcode original durante el fetch del segundo byte, impidiendo que el IR sobrescriba la instrucción en curso. Segundo, los nibbles del segundo byte se capturan en second_byte_reg durante M1 y M2 del segundo ciclo, acumulando la dirección de destino sin alterar ningún registro arquitectónico. Tercero,

la evaluación de la condición es combinacional y continua: el resultado (`jc_n_taken`) está disponible antes de X3 y determina, en ese mismo estado, si la pila escribe la dirección de salto o el incremento normal del contador de programa.

4.10. Limitaciones respecto a la especificación

Tabla 4.21. Funcionalidad no implementada respecto a la especificación original

Característica	Descripción
RAM 4002 e instrucciones	WRM, RDM, WR0–WR3, RD0–RD3, ADM, SBM. La señal <code>cm_ram[3:0]</code> está declarada pero sin lógica de control.
FIN (Fetch Indirect)	Requiere un ciclo de fetch adicional no previsto en la FSM actual.
Grupo OPR completo	WPM, IAC, RAL, RAR, TCC, DCL, KBP. Solo TCS está parcialmente contemplado.
<code>clk_ph2</code>	Declarado en la entidad pero no utilizado en la lógica interna; toda la temporización usa únicamente <code>clk_ph1</code> . En el chip original, $\phi 2$ actuaba como señal de <i>strobe</i> para coordinar la CPU con los chips externos (ROM 4001, RAM 4002): la CPU ponía la dirección en el bus durante $\phi 1$, y los chips externos latchaban esa dirección y respondían durante $\phi 2$. En esta implementación la ROM es interna a la FPGA y recibe directamente la señal <i>fase</i> del bloque Timing & Control, por lo que el <i>handshake</i> de dos fases queda implícito en la máquina de estados y $\phi 2$ resulta innecesario.

5.

Verificación

La verificación se realiza mediante dos testbenches simulados con XSim.

5.1. Testbench del multiplexor de bus interno

Fichero: `tb_mux_interno.vhd`

Verifica el comportamiento del `bus_interno_mux` con todas las combinaciones de señales `_oe`: un único driver activo vuelca su valor al bus; dos drivers simultáneos producen "XXXX"; con todos los `_oe` inactivos el bus toma `BUS_CERO`.

5.2. Testbench del sistema completo

Fichero: `tb_cpu_4004.vhd`

Instancia `cpu_4004_top` junto con la entidad `ROM` cargada con el programa de la calculadora. Aplica reset inicial, genera los dos relojes en cuadratura y deja correr la simulación observando el bus `D0-D3`, las señales `sync` y `cm_rom`, y el contenido del acumulador y el scratchpad en los puntos clave de la ejecución.

6.

Implementación

El diseño ha sido sintetizado e implementado con Vivado sobre la Basys 3 Artix-7 XC7A35TICPG236-1L. Los resultados de utilización de recursos y el informe de timing se incluirán en esta sección.

7.

Conclusiones

Este proyecto ha demostrado la viabilidad de implementar la arquitectura del Intel 4004 en [VHDL](#) sobre una [FPGA](#) moderna, manteniendo correspondencia directa con la especificación original. La adopción de una jerarquía estructural estricta ha facilitado tanto la verificación incremental como la trazabilidad con el diagrama de bloques del chip.

El subconjunto de instrucciones implementado es suficiente para ejecutar la calculadora [BCD](#) de demostración, validando la ruta de datos, la unidad de control, el banco de registros y la interfaz con la memoria de programa.

A.

Listado de ficheros del proyecto

Tabla A.1. Ficheros fuente VHDL del proyecto

Fichero	Descripción
pkg_4004.vhd	Paquete global de constantes
d_ff_en.vhd	FF D con enable (primitiva base)
flipflopJK.vhd	FF JK (puntero de pila)
registro_4b.vhd	Registro de 4 bits
registro_12b.vhd	Registro de 12 bits
accumulator.vhd	Acumulador
temp_register.vhd	Registro temporal
instruction_register.vhd	Registro de instrucción
flag_flip_flops.vhd	Registros de flags
alu.vhd	Unidad Aritmético-Lógica
decimal_adjust.vhd	Ajuste decimal BCD
scratch_pad.vhd	Banco de 16 registros (scratchpad)
mtx_scratchpad.vhd	Multiplexor 16:1 del scratchpad
puntero_stack.vhd	Puntero de pila bidireccional
mtx_stack.vhd	Multiplexor de pila con serialización
stack_3L.vhd	Pila de direcciones de 3 niveles
bus_interno_mux.vhd	Multiplexor central del bus interno
data_bus_buffer.vhd	Buffer triestado del bus externo
instruction_decoder.vhd	Decodificador de instrucciones
timing_and_control.vhd	FSM de temporización y control
cpu_4004_top.vhd	Integrador top-level de la CPU
ROM.vhd	ROM de programa con puertos de E/S
basys3_4004_soc.vhd	SoC para la placa Basys 3
tb_mux_interno.vhd	Testbench del bus interno
tb_cpu_4004.vhd	Testbench del sistema completo
stub.vhd	Componente auxiliar de simulación
basys3.xdc	Constraints de pines para la Basys 3

Índice de términos

Glosario

This document is incomplete. The external file associated with the glossary ‘main’ (which should be called `memoria.gls`) hasn’t been created.

Check the contents of the file `memoria.gls`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

If you don’t want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "memoria"
```

- Run the external (Perl) application:

```
makeglossaries "memoria"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

Siglas

This document is incomplete. The external file associated with the glossary ‘acronym’ (which should be called `memoria.acr`) hasn’t been created.

Check the contents of the file `memoria.acn`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`.
For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "memoria"
```

- Run the external (Perl) application:

```
makeglossaries "memoria"
```

Then rerun $\LaTeX$ on this document.

This message will be removed once the problem has been fixed.

